

DS362 / IT3031 Exam Notes

Lecture-by-lecture notes rewritten from the uploaded slides

March 2026

How to use this note pack

These notes are written so you can study **from the notes instead of from the slides**.

Each lecture is rewritten in an easier structure:

- **Big idea** - what the lecture is really teaching
- **Core concepts** - the definitions and theory you need
- **Patterns** - the repeated logic the lecturer uses
- **Examples** - so the syntax is easier to remember
- **Exam focus** - what you should be able to explain in an answer
- **Common mistakes** - what usually confuses students

Quick map of the lectures

1. **Lecture 1** - Introduction to Object-Relational Database Systems (ORDB)
2. **Lecture 2** - ORDB Collections
3. **Lecture 3** - ORDB Methods and Inheritance
4. **Lecture 4A** - Triggers and ER-to-ORDB Mapping
5. **Lecture 4B** - XML Databases
6. **Lecture 5** - Components of a DBMS, Disks, and Files
7. **Lecture 6** - File Organization and Indexes

One pattern that connects the whole module

This module has **three big layers**:

1. **Data modeling layer** - How to represent more complex data than ordinary relational tables.
This is where ORDB, collections, inheritance, REFs, XML, and ER mapping appear.
2. **Storage layer** - How the DBMS physically stores and moves data.
This is where disks, pages, buffer management, and file organization appear.
3. **Access and performance layer** - How the DBMS finds data efficiently.
This is where indexes, B+ trees, hashing, and query optimization ideas appear.

If you remember that pattern, the lectures become much easier to connect.

Lecture 1 - Introduction to Object-Relational Database Systems

Based on: Introduction to ORDB lecture slides.

1. Big idea

The relational model is excellent for ordinary business data, but it becomes awkward when the data is **complex, nested, multi-valued, or hierarchical**.

An **object-relational database system (ORDBMS)** extends the relational model with selected object-oriented ideas so that we can still use SQL while handling richer data structures.

In one sentence:

ORDB = relational databases + object-oriented features, without throwing away SQL.

2. Why the relational model became limited

The lecture starts by reminding us why the relational model became popular:

- tables are simple
- keys and foreign keys give structure
- SQL is powerful and standard
- relational theory has a strong mathematical base
- it works very well for business and administrative data

But the lecture also shows the limitations.

2.1 Problems with pure relational design

A pure relational model prefers **atomic attributes**. That means each attribute should hold only one value. This creates problems when a real-world thing naturally has:

- **set-valued attributes**
 - an employee may have many qualifications
 - a person may have many phone numbers
- **structured attributes**
 - an address is naturally made of house number, street, city, postcode
- **hierarchies**
 - a student is a person
 - a sales manager is an employee
- **complex data**
 - multimedia, CAD data, networks, documents

The result is that a simple concept from the real world often gets split across many tables, creating many joins.

2.2 Example: person with phone numbers and children

A relational design might force us into something like:

Person(**ID**, Name, PhoneN, ChildID)

This is bad because:

- the key becomes something like (ID, PhoneN, ChildID)
- ID -> Name, so the relation is not in 3NF

- the same person name repeats again and again

To fix normal form issues, we decompose into:

```
Person(ID, Name)
Phone(ID, PhoneN)
ChildOf(ID, ChildID)
```

This is logically cleaner, but now queries become more complicated.

2.3 Why ORDB helps

In ORDB thinking, it is natural to say:

```
Person(
  ID: string,
  Name: string,
  PhoneN: {string},
  Child: {string}
)
```

Now the structure is closer to the real world. The database is allowed to store collections and richer types, so the schema matches the problem better.

3. Historical path: RDB -> OODB -> ORDB

The lecture shows a short history.

3.1 Relational DBMS

The relational model dominated because it offered:

- simplicity
- strong theory
- standard query language
- very good support for business data

3.2 Object-oriented DBMS

Object-oriented database models were proposed to handle:

- complex objects
- nested data
- object identity
- inheritance
- methods

They were powerful in theory, but they never replaced relational databases in mainstream use.

3.3 Object-relational DBMS

ORDBMSs became the compromise:

- keep the relational foundation
- add object features where useful

- extend SQL rather than replace it
- support richer applications

This is why the lecture says ORDB is a **bridge between relational and object-oriented models**.

4. Main SQL extensions for ORDB

The lecture highlights the main SQL/Oracle extensions.

4.1 User-defined types (UDTs)

A **user-defined type** is like a class definition. It can contain:

- attributes
- methods

In Oracle, the syntax is:

```
CREATE TYPE BarType AS OBJECT (
  name CHAR(20),
  addr CHAR(20)
);
/
```

Important idea:

- creating a type does **not** create a table
- it only defines a structure

4.2 Two uses of a UDT

A UDT can be used:

1. **as a row type** of a table
2. **as the type of a column** in a relational table

That means object types can appear either at the whole-row level or inside columns.

4.3 Methods

A type can have methods. The later lecture goes deeper, but even here the important point is:

- ORDB types are not just data containers
- they can also store behavior

4.4 REF and Deref

ORDB introduces **references** between objects.

If T is a type, then REF T is a reference to an object of type T.

This gives object-style links inside the database.

4.5 Inheritance

Types can inherit from other types.

That allows modeling hierarchies such as:

- Person
- Student UNDER Person
- Employee UNDER Person

5. Object types in Oracle

The lecture uses Oracle syntax. The core pattern is:

```
CREATE TYPE TypeName AS OBJECT (
  attribute1 datatype,
  attribute2 datatype
);
/
```

The slash / is important in Oracle tools because it finalizes the stored type definition.

6. Row objects vs column objects

This is one of the most exam-friendly distinctions in the lecture.

6.1 Row objects

If we create a table **of** a type, each row is an object of that type.

```
CREATE TABLE bars OF BarType;
```

Meaning:

- bars is an **object table**
- each row is a BarType object

Why this matters

The whole row behaves like one object, so methods and object references fit naturally.

6.2 Column objects

We can also use object types as column types.

```
CREATE TYPE BeerType AS OBJECT (
  name CHAR(20),
  manf CHAR(20)
);
/

CREATE TABLE menu (
  bar BarType,
  beer BeerType,
  price REAL
);
```

Meaning:

- menu is still a relational table
- but some columns contain object values

6.3 Difference between the two

Idea	Row object / object table	Column object
Where the type is used	Whole row	Individual column
Example	CREATE TABLE bars OF BarType	bar BarType in a table
Row identity	Row itself is an object	Row is relational, only some columns are objects
Best mental model	Object table	Relational table with rich attributes

Exam wording to remember:

- *Object table* -> table of a user-defined object type
- *Column object* -> object stored inside a column of a regular table

7. Inserting and selecting object values

7.1 Insert as ordinary column values

Because the object has two attributes, Oracle lets us insert like this:

```
INSERT INTO bars VALUES ('Sally's', 'River Rd');
```

7.2 Insert using the type constructor

Every object type has a constructor with the same name as the type.

```
INSERT INTO bars VALUES (  
  BarType('Joe's Bar', 'Maple St.')  
);
```

This is usually the clearer object-style syntax.

7.3 Select normally

```
SELECT * FROM bars;
```

This returns rows as if they were a normal two-column table.

7.4 Select the whole object

```
SELECT VALUE(b)  
FROM bars b;
```

VALUE(b) means: return the entire object value represented by alias b.

8. REF and Deref

This is one of the most important ORDB ideas.

8.1 What is a REF?

A REF is like a database pointer to an object.

Example type:

```
CREATE TYPE MenuType2 AS OBJECT (
  bar REF BarType,
  beer REF BeerType,
  price FLOAT
);
/
```

Here:

- bar points to a BarType object
- beer points to a BeerType object

8.2 How to obtain a REF

You get a reference by selecting a row object with the REF operator.

```
INSERT INTO sells VALUES (
  (SELECT REF(b) FROM bars b WHERE name = 'Jim's'),
  (SELECT REF(e) FROM beers e WHERE name = 'Swan'),
  2.40
);
```

Pattern:

REF(alias) -> gives a reference to that object-table row

8.3 Dereferencing

Dereferencing means following the pointer to reach the target object.

In Oracle, this is often automatic through the dot operator.

```
SELECT s.bar.name, s.beer.name, s.price
FROM sells s;
```

Even though bar and beer are references, Oracle automatically follows them.

This is often described as an **implicit join**.

8.4 Alias rule

The lecture explicitly warns that aliases are required.

This works:

```
SELECT s.beer.name
FROM sells s;
```

These do **not** work correctly:

```
SELECT beer.name FROM sells;
SELECT sells.beer.name FROM sells;
```

Exam memory trick: when accessing object attributes, always think **alias dot attribute**.

9. Another REF example: person and manager

```
CREATE TYPE person AS OBJECT (
  name VARCHAR2(30),
```

```

manager REF person
);
/

CREATE TABLE person_table OF person;

SELECT x.name, x.manager.name
FROM person_table x;

```

This is a good example because it shows a self-reference:

- a person points to another person
- x.manager.name follows that REF and retrieves the manager's name

10. Scoped REFs

The lecture also shows **scoped REFs**.

```

CREATE TYPE dept_t AS OBJECT (
  dno INTEGER,
  dname VARCHAR(12)
);
/

CREATE TABLE dept_table OF dept_t;

CREATE TABLE dept_loc (
  dept REF dept_t REFERENCES dept_table,
  loc VARCHAR(20)
);

```

Meaning:

- dept can only reference objects from dept_table
- this gives more control and integrity

11. Co-dependent types and incomplete types

Sometimes two types need to refer to each other.

Example:

- employee has a REF to department
- department has a REF to employee (manager)

This creates a circular dependency.

Oracle solves this by allowing an **incomplete type** first.

```

CREATE TYPE department;
/

```

Then another type can reference it before the full definition exists.

This is important because otherwise neither type could be declared first.

12. Constraints on object tables

Object tables can still use familiar constraints.

```
CREATE TYPE person AS OBJECT (  
  pid NUMBER,  
  name VARCHAR(25),  
  address VARCHAR(50)  
);  
/  
  
CREATE TABLE person_table OF person (  
  pid PRIMARY KEY,  
  name NOT NULL  
);
```

So object-relational does **not** mean abandoning relational discipline.

13. NULL objects and attributes

The lecture makes an important conceptual point:

A NULL can represent an unknown value at many levels:

- ordinary column value
- whole object value
- attribute inside an object
- collection or collection element

So nullability still matters in ORDB, just like in ordinary relational design.

14. RDB vs OODB vs ORDB - easy comparison

Model	Main strength	Main weakness	Best way to remember
RDB	Simple tables, SQL, strong theory	Awkward for complex/nested data	Great for classic business data
OODB	Rich objects, inheritance, methods	Limited adoption, less standardization	Strong object idea, weaker mainstream adoption
ORDB	Keeps SQL and tables, adds object features	More complex than plain relational design	Middle path between the two

15. Exam focus

You should be able to explain:

1. **Why ORDB was needed**
2. **How user-defined types extend SQL**
3. **Difference between row objects and column objects**
4. **What REF/DEREF means**

5. How ORDB reduces schema awkwardness for complex data

16. Common mistakes

- Thinking ORDB replaces relational DBs completely - it does not
- Thinking CREATE TYPE automatically creates a table - it does not
- Forgetting that REF acts like a pointer to an object table row
- Forgetting to use aliases when accessing object attributes
- Mixing up object tables with ordinary tables that only contain object columns

17. One final memory sentence

Lecture 1 teaches that ORDB lets us model rich real-world data while still staying inside the SQL world.

Lecture 2 - ORDB Collections

Based on: ORDB Collections lecture slides.

1. Big idea

Collections allow a single row or object to contain **multiple related values**.

This is useful when a real-world entity naturally has a one-to-many relationship attached to it.

Typical examples from the lecture:

- an investor has many share purchases
- an employee has many projects
- a manufacturer has many beers

The lecture focuses on two Oracle collection types:

- **VARRAYs**
- **Nested tables**

2. Why collections matter

In plain relational design, one-to-many information is usually stored in a separate table. That is still valid, but ORDB allows another option:

- keep the related values together as part of the object
- query and manipulate them as a collection

This is useful when the collection is logically part of the object itself.

3. VARRAYs

3.1 Definition

A **VARRAY** is a variable-size array with a **maximum size fixed at declaration time**.

Example:

```
CREATE TYPE price_arr AS VARRAY(10) OF NUMBER(12,2);  
/
```

Meaning:

- the type can hold up to 10 values
- each value is NUMBER(12,2)

3.2 Main characteristics of VARRAYs

- ordered collection
- maximum size must be declared
- good for small bounded lists
- type declaration itself does not allocate storage

A useful way to think about a VARRAY is:

a list with order, size limit, and object-like behavior

3.3 Where a VARRAY can be used

A VARRAY can be:

- the datatype of a column in a relational table
- the datatype of an attribute in an object type

Example from the lecture:

```
CREATE TYPE exchararray AS VARRAY(5) OF VARCHAR(12);  
/  
  
CREATE TYPE share_t AS OBJECT (  
  cname  VARCHAR(12),  
  cprice NUMBER(6,2),  
  exchanges exchararray,  
  dividend NUMBER(4,2),  
  earnings NUMBER(6,2)  
);  
/
```

Here, exchanges is a VARRAY attribute.

3.4 Inserting a VARRAY

You use the collection constructor.

```
CREATE TABLE shares OF share_t (  
  cname PRIMARY KEY  
);  
  
INSERT INTO shares VALUES (  
  'BHP',  
  10.50,  
  exchararray('Sydney', 'New York'),  
  1.50,
```

```
3.20
```

```
);
```

Pattern to remember:

```
collection_type_name(value1, value2, ...)
```

3.5 Another VARRAY example

```
CREATE TYPE price_arr AS VARRAY(10) OF NUMBER(12,2);
/

CREATE TABLE pricelist (
  pno INTEGER,
  prices price_arr
);

INSERT INTO pricelist VALUES (
  1,
  price_arr(2.50, 3.75, 4.25)
);
```

3.6 Retrieving values from a VARRAY

A plain select shows the collection as one value.

```
SELECT * FROM pricelist;
```

Output idea:

```
1 PRICE_ARR(2.5, 3.75, 4.25)
```

If you want each element separately, you convert the collection to a table-like form using TABLE(...).

```
SELECT pno, s.COLUMN_VALUE AS price
FROM pricelist p, TABLE(p.prices) s;
```

Important point: COLUMN_VALUE

When the collection holds scalar values rather than object values, Oracle uses the pseudocolumn COLUMN_VALUE.

4. Nested tables

4.1 Definition

A **nested table** allows an attribute to contain a relation-like set of rows.

If T is a user-defined type, then:

```
CREATE TYPE S AS TABLE OF T;
```

means S is a collection type whose elements are objects of type T.

4.2 Example: Beer manufacturers and beers

```
CREATE TYPE BeerType AS OBJECT (
  name CHAR(20),
  kind CHAR(10),
```

```

colour CHAR(10)
);
/

CREATE TYPE BeerTableType AS TABLE OF BeerType;
/

```

Now use it in a table:

```

CREATE TABLE Manfs (
  name CHAR(30),
  addr CHAR(50),
  beers BeerTableType
)
NESTED TABLE beers STORE AS beer_table;

```

Meaning:

- each manufacturer row has a nested table called beers
- Oracle stores nested-table rows in a separate storage table called beer_table

4.3 Key idea: logical vs physical view

Logically, each row seems to have its own table of beers.

Physically, Oracle stores all nested-table elements for one attribute in a single storage structure, while preserving which parent row each nested table belongs to.

So in exams you can say:

A nested table behaves like a relation-valued attribute, but Oracle stores it in an underlying storage table.

4.4 Retrieving a nested table as one value

```

SELECT beers
FROM Manfs
WHERE name = 'Anheuser-Busch';

```

This returns one value that looks like a collection constructor containing BeerType(...) objects.

4.5 Querying inside a nested table

To query individual nested rows, use TABLE(...).

```

SELECT b.name
FROM TABLE(
  SELECT beers
  FROM Manfs
  WHERE name = 'Anheuser-Busch'
) b
WHERE b.kind = 'ale';

```

Pattern:

```
TABLE(subquery returning a collection)
```

That is how you turn a collection into a relation-like form in SQL.

5. Nested table example with employees and projects

The lecture gives a clean example:

```
CREATE TYPE proj_t AS OBJECT (  
  projno NUMBER,  
  projname VARCHAR(15)  
);  
/  
  
CREATE TYPE proj_list AS TABLE OF proj_t;  
/  
  
CREATE TYPE employee_t AS OBJECT (  
  eno NUMBER,  
  projects proj_list  
);  
/  
  
CREATE TABLE employees OF employee_t (  
  eno PRIMARY KEY  
)  
  
NESTED TABLE projects STORE AS employees_proj_table;
```

Insert a row:

```
INSERT INTO employees VALUES (  
  1000,  
  proj_list(  
    proj_t(101, 'Avionics'),  
    proj_t(102, 'Cruise control')  
  )  
);
```

Retrieve the projects of employee 1000:

```
SELECT *  
FROM TABLE(  
  SELECT t.projects  
  FROM employees t  
  WHERE t.eno = 1000  
);
```

6. Collection unnesting

A very important pattern is **unnesting**.

Unnesting means flattening the collection and joining each collection element with its parent row.

```
SELECT e.eno, p.*  
FROM employees e, TABLE(e.projects) p;
```

This gives one row per project per employee.

Easy meaning

If employee 1000 has 2 projects, that single employee row becomes 2 result rows after unnesting.

7. DML on collections

The lecture is important because it shows that collections are not read-only. We can also modify them.

7.1 Insert into a nested collection

```
INSERT INTO TABLE(  
  SELECT e.projects  
  FROM employees e  
  WHERE e.eno = 1000  
)  
VALUES (103, 'Project Neptune');
```

7.2 Update a nested collection row

```
UPDATE TABLE(  
  SELECT e.projects  
  FROM employees e  
  WHERE e.eno = 1000  
) p  
SET p.projname = 'Project Pluto'  
WHERE p.projno = 103;
```

7.3 Delete from a nested collection

```
DELETE TABLE(  
  SELECT e.projects  
  FROM employees e  
  WHERE e.eno = 1000  
) p  
WHERE p.projno = 103;
```

Pattern to remember: when changing nested-table elements, the TABLE(...) expression identifies the collection to manipulate.

8. NULL collection vs empty collection

This is a classic exam trap.

8.1 Set collection to NULL

```
UPDATE employees e  
SET e.projects = NULL  
WHERE e.eno = 1000;
```

Meaning:

- the collection value is unknown or absent
- not just empty

8.2 Empty collection constructor

```
UPDATE employees e  
SET e.projects = proj_list()  
WHERE e.eno = 1000;
```

Meaning:

- the collection exists
- but it currently contains zero rows

8.3 Why the difference matters

State	Meaning
NULL	no collection value / unknown
proj_list()	collection exists but is empty

That difference affects logic, querying, and later updates.

9. Multilevel collections

The lecture ends by showing that collection elements can themselves contain collections.

Possible combinations include:

- nested table of nested table
- nested table of varray
- varray of nested table
- varray of varray
- user-defined types whose attributes are collections

Example idea from the lecture

- a star has many planets
- each planet has many satellites

So we might define:

```
sat_t    -> one satellite
sat_ntt  -> nested table of satellites
planet_t -> has a satellites attribute of type sat_ntt
planet_ntt -> nested table of planets
star_t   -> has a planets attribute of type planet_ntt
```

And then the table declaration needs nested-table storage for both levels.

This shows that Oracle collections can model deep hierarchies.

10. VARRAY vs nested table

This comparison is extremely important.

Feature	VARRAY	Nested table
Order	Preserved	Can be treated more like a set/table
Maximum size	Fixed maximum declared	No fixed maximum in the type definition
Typical use	Small bounded list	Larger relation-like child set

Feature	VARRAY	Nested table
Querying	Can unnest with TABLE(...)	Can unnest with TABLE(...)
Storage idea	More array-like	More table-like
Best example	list of exchanges, small list of values	projects of employee, beers of manufacturer

Easy memory trick

- **VARRAY** -> "small ordered list"
- **Nested table** -> "child table inside a row"

11. When to choose which

Choose a **VARRAY** when:

- the number of elements is naturally limited
- order matters
- the list is conceptually compact

Choose a **nested table** when:

- the number of elements can grow
- you want table-like querying and manipulation
- the child rows feel like a small relation attached to the parent

12. Exam focus

Be able to explain:

1. what a collection type is
2. how VARRAY and nested table differ
3. how to declare, insert, and query collections
4. what TABLE(...) does
5. why NULL and empty collection are different
6. how multilevel collections work conceptually

13. Common mistakes

- Forgetting to use the constructor when inserting collection values
- Forgetting the NESTED TABLE ... STORE AS ... clause
- Confusing scalar VARRAY elements with object-type nested-table elements
- Not knowing that TABLE(...) is used for querying or modifying collection contents
- Treating NULL and empty collection as the same thing

14. One final memory sentence

Lecture 2 teaches how ORDB stores one-to-many information directly inside rows and objects using collections.

Lecture 3 - ORDB Methods and Inheritance

Based on: ORDB Methods and Inheritance lecture slides.

1. Big idea

Lecture 1 introduced object types as structures. Lecture 3 adds the rest of the object-oriented flavor:

- **methods** -> behavior attached to types
- **comparison methods** -> how objects can be compared
- **inheritance** -> type hierarchies and specialization

So the core message is:

ORDB objects are not just stored structures; they can also have behavior and belong to type hierarchies.

2. Methods in object types

A method is a function or procedure declared in a type definition.

2.1 Why methods matter

Methods let us keep logic close to the data it belongs to.

Example idea:

- if a MenuType object has a price, it can also have a method that converts that price into another currency

This is **encapsulation**: data and operations are grouped together.

2.2 Declaration and definition

In Oracle, methods are:

- **declared** in CREATE TYPE
- **defined** in CREATE TYPE BODY

2.3 Example of member method declaration

```
CREATE TYPE MenuType AS OBJECT (  
  bar REF BarType,  
  beer REF BeerType,  
  price FLOAT,  
  MEMBER FUNCTION priceInYen(rate IN FLOAT)  
    RETURN FLOAT  
);  
/
```

2.4 Type body

```
CREATE TYPE BODY MenuType AS  
  MEMBER FUNCTION priceInYen(rate FLOAT)  
    RETURN FLOAT IS  
  BEGIN  
    RETURN rate * SELF.price;  
  END;
```

```
END;  
END;  
/
```

3. SELF in methods

SELF is the current object instance on which the method is being called.

In the example:

- SELF.price means the price attribute of the current MenuType object

Oracle often lets you omit SELF, so price alone may work inside the method body. But conceptually, SELF is very important.

Easy way to think about SELF

If this were object-oriented programming, SELF is like this in some other languages.

4. Member method rules to remember

The lecture gives several useful reminders:

- some methods take parameters, some do not
- if a method has no arguments, method invocation still uses parentheses in SQL, e.g. e.age()
- the type body must define all declared methods
- multiple methods can appear in one type body, separated properly

5. Adding a new method later

The lecture shows ALTER TYPE.

```
ALTER TYPE MenuType  
ADD MEMBER FUNCTION priceInUSD(rate FLOAT)  
RETURN FLOAT CASCADE;
```

Then the type body must be recreated or replaced with the implementation of all current methods.

Why CASCADE matters

When a type changes, dependent objects may also need to be updated. CASCADE tells Oracle to propagate the change.

6. Using methods in SQL

Once defined, methods can be called on object instances.

```
SELECT s.beer.name, s.priceInYen(106.0)  
FROM Sells s  
WHERE s.bar.name = 'Joe's Bar';
```

Pattern:

```
alias.methodName(arguments)
```

So methods become part of SQL expressions.

7. Comparison of objects

Ordinary scalar values like NUMBER, DATE, and CHAR already have built-in comparison rules.

Objects do not.

That creates a problem:

- how do we sort objects?
- how do we use DISTINCT or ORDER BY on them?
- how do we evaluate `obj1 > obj2`?

Oracle solves this with **map methods** or **order methods**.

8. Map methods

8.1 Idea

A map method converts an object to a scalar value, and Oracle compares the scalar values instead.

Example idea

For rectangles, compare by area.

If area is `length * width`, then object comparison becomes scalar comparison on that area.

8.2 Properties of a map method

- parameterless member function
- declared with the keyword MAP
- returns a scalar type such as NUMBER, DATE, VARCHAR2
- called automatically when comparison is needed

Example

```
CREATE TYPE Rectangle_type AS OBJECT (  
  length NUMBER,  
  width  NUMBER,  
  MAP MEMBER FUNCTION area RETURN NUMBER  
);  
/  
  
CREATE TYPE BODY Rectangle_type AS  
  MAP MEMBER FUNCTION area RETURN NUMBER IS  
  BEGIN  
    RETURN length * width;  
  END area;  
END;  
/
```

Now two rectangle objects can be compared by area.

Why DISTINCT worked in the lecture example

If (1,2) and (2,1) both have area 2, they map to the same scalar value for comparison purposes.

9. Order methods

9.1 Idea

An order method directly compares one object with another object of the same type.

Instead of mapping both objects to scalars, it returns:

- < 0 if SELF is less than the other object
- 0 if they are equal
- > 0 if SELF is greater

9.2 Example

```
CREATE TYPE Customer_typ AS OBJECT (  
  id NUMBER,  
  name VARCHAR2(20),  
  addr VARCHAR2(30),  
  ORDER MEMBER FUNCTION match(c Customer_typ)  
    RETURN INTEGER  
);  
/
```

Implementation:

```
CREATE TYPE BODY Customer_typ AS  
  ORDER MEMBER FUNCTION match(c Customer_typ)  
    RETURN INTEGER IS  
  BEGIN  
    IF id < c.id THEN  
      RETURN -1;  
    ELSIF id > c.id THEN  
      RETURN 1;  
    ELSE  
      RETURN 0;  
    END IF;  
  END;  
END;  
/
```

This compares customers by customer ID.

10. Map vs order method

This difference is very exam-friendly.

Feature	Map method	Order method
Basic idea	Object -> scalar -> compare scalars	Compare one object directly with another
Parameters	No explicit parameter	One parameter of same type
Return type	Scalar type	Integer-like negative/zero/positive

Feature	Map method	Order method
Efficiency for sorting large sets	Usually better	Usually less efficient
Best when	A simple scalar representation exists	Comparison logic is more complex

Important rule

A type can have:

- **one map method, or**
- **one order method**

but **not both**.

If neither exists

Oracle can still test only simple equality/inequality based on corresponding attribute values, but advanced ordering behavior is not available.

11. Methods on nested tables

The lecture also shows that a method can operate on a collection attribute inside the object.

Example type:

```
CREATE TYPE proj_t AS OBJECT (
  projno NUMBER,
  projname VARCHAR(15)
);
/

CREATE TYPE proj_list AS TABLE OF proj_t;
/

CREATE TYPE emp_t AS OBJECT (
  eno NUMBER,
  projects proj_list,
  MEMBER FUNCTION projcnt RETURN INTEGER
);
/
```

Body:

```
CREATE OR REPLACE TYPE BODY emp_t AS
  MEMBER FUNCTION projcnt RETURN INTEGER IS
    pcount INTEGER;
  BEGIN
    SELECT COUNT(p.projno)
    INTO pcount
    FROM TABLE(self.projects) p;
    RETURN pcount;
  END;
END;
/
```

Why this is important

This shows how a method can hide internal details.

Instead of writing the count query every time, we just call:

```
SELECT e.eno, e.projcnt() AS projcount  
FROM emptab e;
```

12. Inheritance

Inheritance is the second half of this lecture.

12.1 Basic idea

A subtype automatically gets the attributes and methods of its supertype.

This is useful because:

- common structure is not repeated
- common behavior is shared
- specialization becomes easier

12.2 Example hierarchy

The lecture shows a person hierarchy with types such as:

- Person
- Student
- Office worker
- Sales person
- Engineer
- Secretary
- Graduate
- UnderGrad
- Project manager
- Sales manager
- District manager

The exact tree is less important than the idea:

subtypes represent more specialized versions of the supertype

13. FINAL and NOT FINAL

By default, Oracle object types are **FINAL**.

That means:

- no subtype can be created under them

To allow subtypes, the type must be declared NOT FINAL.

Example

```
CREATE TYPE Person_type AS OBJECT (  
  pid  NUMBER,  
  name  VARCHAR2(30),  
  address VARCHAR2(100)  
) NOT FINAL;  
/
```

Now subtypes can be defined under Person_type.

14. Creating subtypes with UNDER

Example:

```
CREATE TYPE Student_type UNDER Person_type (  
  deptid NUMBER,  
  major  VARCHAR2(30)  
) NOT FINAL;  
/
```

Meaning:

- Student_type inherits pid, name, and address
- it adds deptid and major

Key rules

A subtype can:

- add new attributes
- add new methods
- override inherited methods

A subtype cannot:

- remove inherited attributes
- change the datatype of inherited attributes

15. Multiple child types and multi-level inheritance

The lecture shows that one parent can have many children.

Example:

```
CREATE TYPE Employee_type UNDER Person_type (  
  empid NUMBER,  
  mgr  VARCHAR2(30)  
);  
/
```

And a subtype can itself have subtypes:

```
CREATE TYPE PartTimeStudent_type UNDER Student_type (  
  numhours NUMBER  
);  
/
```


So Oracle supports hierarchical type trees, not just one level.

16. Table of a supertype

A very important idea: a table of a supertype can store subtype instances.

```
CREATE TABLE person_tab OF person_type (  
  pid PRIMARY KEY  
);
```

Now we can insert a Student_type object into person_tab:

```
INSERT INTO person_tab VALUES (  
  student_type(  
    4,  
    'Edward Learn',  
    '65 Marina Blvd, Ocean Surf, WA, 6725',  
    40,  
    'CS'  
  )  
);
```

This works because a student is a person.

17. Querying hierarchies

17.1 Select all instances

```
SELECT VALUE(p)  
FROM person_tab p;
```

This returns all rows, including subtype objects stored in the supertype table.

17.2 Select instances of a type and its subtypes

```
SELECT VALUE(s)  
FROM person_tab s  
WHERE VALUE(s) IS OF (Student_type);
```

This returns:

- students
- and subtypes of students, such as part-time students

17.3 Select only a particular type, excluding subtypes

```
SELECT VALUE(s)  
FROM person_tab s  
WHERE VALUE(s) IS OF (ONLY Student_type);
```

This returns only exact Student_type objects.

17.4 Access subtype attributes with TREAT

```
SELECT name,  
  TREAT(VALUE(p) AS PartTimeStudent_type).numhours AS hours  
FROM person_tab p  
WHERE VALUE(p) IS OF (ONLY PartTimeStudent_type);
```

Why TREAT is needed:

- numhours exists only in the subtype
- Oracle must be told to treat the object as that subtype

18. NOT INSTANTIABLE types and methods

This lecture also introduces a very OO-style idea.

18.1 Not instantiable type

A type can be declared:

- NOT INSTANTIABLE
- NOT FINAL

This means:

- you cannot create objects directly of that type
- it is meant only as a supertype

Example idea:

```
CREATE TYPE Address_typ AS OBJECT (...) NOT INSTANTIABLE NOT FINAL;
```

That is similar to an abstract class in programming.

18.2 Not instantiable method

A method can be declared without implementation in the supertype, forcing subtypes to provide the implementation.

This is useful when:

- every subtype should implement the method differently

If a subtype fails to implement inherited non-instantiable methods, that subtype must also remain not instantiable.

19. FINAL methods, overriding, and overloading

19.1 Final method

If a method is declared FINAL, subtypes cannot override it.

Important contrast:

- types are final by default
- methods are **not** final by default

19.2 Overriding

Overriding means a subtype provides a new implementation of an inherited method.

Example idea:

- supertype Shape defines calculate_area()
- subtype Rectangle overrides it one way
- subtype Circle overrides it another way

19.3 Overloading

Overloading means two methods have the same name but different signatures.

Example from the lecture idea:

```
MEMBER FUNCTION fun(x NUMBER)
MEMBER FUNCTION fun(x DATE)
```

Same method name, different parameter types.

20. FINAL vs NOT FINAL, overriding vs overloading

Concept pair	Meaning
FINAL type	cannot have subtypes
NOT FINAL type	may have subtypes
overriding	subtype replaces inherited implementation
overloading	same method name, different parameter list/signature

21. Exam focus

Be able to explain:

1. how methods are declared and defined in Oracle
2. how SELF works
3. how object comparison is supported using map/order methods
4. how inheritance works using NOT FINAL and UNDER
5. how subtype objects are stored and queried in a supertype table
6. the meaning of IS OF, ONLY, and TREAT
7. the difference between overriding and overloading

22. Common mistakes

- Thinking methods are defined only in the type body, forgetting they must also be declared in the type specification
- Confusing map and order methods
- Forgetting that only one of map/order can be defined
- Forgetting that types are final by default
- Mixing up IS OF (Type) and IS OF (ONLY Type)
- Using a subtype attribute without TREAT
- Confusing overriding with overloading

23. One final memory sentence

Lecture 3 teaches how ORDB adds behavior and type hierarchies to data definitions.

Lecture 4A - Triggers and ER-to-ORDB Mapping

Based on: Triggers and ER-ORDB Mapping lecture slides.

1. Big idea

This lecture has two connected parts:

1. **Triggers** - automatic actions stored in the database
2. **Mapping ER design into ORDB structures** - how conceptual design becomes object-relational schema

The combined lesson is:

ORDB is not only about richer types; it is also about how we design and automate database behavior.

2. Triggers

2.1 What is a trigger?

A trigger is a stored procedure that runs automatically when a specified event happens.

The lecture mentions two broad groups:

- **DML triggers** - fired by INSERT, UPDATE, DELETE on tables or views
- **System triggers** - fired by system or schema/database events

The focus here is on DML triggers.

2.2 Why triggers are useful

Triggers are typically used for:

- auditing
- enforcing business rules
- maintaining derived data
- recording changes automatically
- blocking invalid actions before they happen

2.3 Triggers on object tables

The lecture emphasizes that triggers can be defined on **object tables** just like ordinary tables.

Example setup:

```
CREATE TYPE location AS OBJECT (  
  campus VARCHAR2(20),  
  building NUMBER,  
  room NUMBER
```

```
);
/

CREATE TYPE staff AS OBJECT (
  sid NUMBER,
  name VARCHAR2(100),
  address VARCHAR2(100),
  office location
);
/

CREATE TABLE staff_tab OF staff (
  sid PRIMARY KEY
);
```

This is important because it shows that ORDB structures still support normal database mechanisms like triggers.

3. Trigger example from the lecture

The lecture then shows a tracking table and a trigger.

```
CREATE TABLE movement (
  sid NUMBER,
  old_office location,
  new_office location
);

CREATE TRIGGER trig1
BEFORE UPDATE OF office ON staff_tab
FOR EACH ROW
WHEN (new.office.campus = 'Bentley')
BEGIN
  IF :new.office.building = 314 THEN
    INSERT INTO movement
    VALUES (:old.sid, :old.office, :new.office);
  END IF;
END;
```

3.1 What this trigger means in plain English

Before a staff member's office is updated:

- only consider rows where the new office campus is Bentley
- if the new office building is 314
- insert a record into movement
- record the staff ID, the old office, and the new office

So the database automatically logs certain office movements.

3.2 Key syntax pieces

Part	Meaning
BEFORE UPDATE OF office ON staff_tab	fire before the office attribute is updated
FOR EACH ROW	row-level trigger, once per affected row
WHEN (...)	extra condition to decide whether the trigger runs
:old	old row values
:new	new row values

3.3 Why the trigger is a good ORDB example

Because office is itself an object of type location, the trigger accesses nested attributes like:

- new.office.campus
- new.office.building

This shows that triggers can work with structured object attributes, not just flat columns.

4. Why ER design needs extension for ORDB

Traditional ER design is usually mapped into purely relational tables. But ORDB gives extra choices.

The lecture says ER models may need to represent:

- **multi-valued attributes**
- **structured-type attributes**
- **collection-type attributes**

Examples:

- store locations of a branch
- address with house number, street, suburb
- list of dates or other grouped values

That means the ER-to-schema mapping can be richer than ordinary table decomposition.

5. Basic mapping rule: entity -> object type -> table

The lecture gives a simple pattern.

Mapping pattern

1. map each ER entity type to an ORDB object type
2. include all attributes of the ER entity in the object type
3. create a table for the type
4. declare key attributes as primary keys
5. map subtypes using inheritance

Example: Employee entity

ER idea:

- Employees(eno, name, lot)

Object-relational mapping:

```
CREATE TYPE Emp_t AS OBJECT (  
  eno CHAR(11),  
  name CHAR(20),  
  lot INTEGER  
);  
/  
  
CREATE TABLE employees OF Emp_t (  
  eno PRIMARY KEY  
);
```

6. Mapping ISA hierarchies

ISA hierarchies map naturally to type inheritance.

Example from the lecture:

- Employees
- Hourly_Emps
- Contract_Emps

Mapping:

```
CREATE TYPE emp_t AS OBJECT (...) NOT FINAL;  
/  
  
CREATE TYPE hourly_emps_t UNDER emp_t (  
  hourly_wages NUMBER(6,2),  
  hours_wkd NUMBER(4,2)  
);  
/  
  
CREATE TYPE contract_emps_t UNDER emp_t (  
  contractid CHAR(10)  
);  
/
```

Key idea

- supertype must be NOT FINAL
- subtypes inherit common attributes automatically

7. Mapping relationships using REFs

The lecture explains how binary relationships are often mapped by storing references.

7.1 Binary relationship with a key constraint

Suppose employees work in departments, and each employee works in one department.

Then the employee type can contain a reference to the department.

```
CREATE TYPE Emp_t AS OBJECT (  
  eno CHAR(11),  
  name CHAR(20),  
  lot INTEGER,  
  workdept REF dept_t  
);  
/
```

Interpretation

If the relationship has a key constraint (for example many employees -> one department), a REF attribute on the many side is a natural choice.

7.2 Relationship with relationship attributes

For a Manages relationship that also has since, the department object may contain:

- a REF to the managing employee
- the since date

Example pattern from the lecture:

```
CREATE TYPE Dept_t AS OBJECT (  
  did INTEGER,  
  dname CHAR(20),  
  budget REAL,  
  mgr REF emp_t,  
  since DATE  
);  
/
```

This is how key constraints and relationship attributes can be stored together.

8. Participation constraints

The lecture shows that participation constraints can be enforced using NOT NULL and reference constraints.

```
CREATE TABLE Department OF Dept_t (  
  did PRIMARY KEY,  
  mgr NOT NULL REFERENCES Employees  
);
```

Meaning

If the relationship says every department must have a manager, then:

- the mgr reference cannot be null

This is a practical way to enforce total participation.

9. Mapping M:N relationships

For many-to-many relationships, a separate type/table is usually best.

Example:

```
CREATE TYPE WorksIn_t AS OBJECT (  
  emp REF emp_t,  
  dept REF dept_t,  
  since DATE  
);  
/  
  
CREATE TABLE worksIn OF WorksIn_t (  
  emp REFERENCES Employees,  
  dept REFERENCES Departments  
);
```

Pattern to remember

If a relationship:

- is M:N, or
- has its own attributes

then make it a separate object type/table.

10. Weak entities

The lecture gives several options for weak entities.

10.1 Option 1 - map like a regular entity

This is the ordinary safe option.

10.2 Option 2 - use nested collection if the weak entity depends only on its owner

This works when:

- the weak entity only participates in its identifying relationship
- it is not independently referenced elsewhere

Example from the lecture:

```
CREATE TYPE depend_t AS OBJECT (  
  pname VARCHAR(12),  
  age INTEGER,  
  policyCost NUMBER(6,2)  
);  
/  
  
CREATE TYPE policy_t AS TABLE OF depend_t;  
/  
  
CREATE TYPE emp_t AS OBJECT (  
  ...,  
  dependents policy_t  
);  
/
```

Interpretation

A dependent is stored as part of the employee object, rather than as a separate independently referenced entity.

10.3 When nested collection is a good weak-entity mapping

Use it when the weak entity:

- belongs naturally inside the owner
- has no separate identity outside the owner
- will not be referenced by other object types

11. Mapping N-ary relationships

For relationships with more than two participating entity types, the lecture recommends a separate type with references to all participants.

Example pattern:

```
CREATE TYPE Contract_t AS OBJECT (  
  part   REF part_t,  
  supplier REF supplier_t,  
  dept   REF dept_t,  
  qty    NUMBER(6,2)  
);  
/
```

This is the object-relational equivalent of a separate relationship table in relational design.

12. Cyber Shop example - full mapping pattern

The lecture ends with a full example involving:

- Customer
- Purchase Order
- Line Item
- Stock Item
- Book and Software as subtypes of Stock Item

This example is extremely useful because it combines almost every mapping idea.

12.1 Regular entities

Map to object types:

- Customer_typ
- StockItem_typ
- PurchaseOrder_typ

12.2 Inheritance

StockItem_typ is declared NOT FINAL, then specialized:

- Book_typ UNDER StockItem_typ
- Software_typ UNDER StockItem_typ

12.3 Weak entity

LineItem_typ is treated as weak and stored as a nested table inside purchase orders.

```
PurchaseOrder_typ(
  PONo,
  ...,
  LineItemList_ntab LineItemList_ntabtyp,
  Cust_ref REF Customer_typ
)
```

12.4 Binary relationships with key constraints

- purchase order references customer via Cust_ref
- line item references stock item via Stock_ref

12.5 Final tables and constraints

The lecture shows object tables like:

- Customers OF Customer_typ (custNo PRIMARY KEY)
- Stocks OF StockItem_typ (stockNo PRIMARY KEY)
- Orders OF PurchaseOrder_typ (PONo PRIMARY KEY, Cust_ref REFERENCES Customers)

and the nested table for line items stored separately.

Why this example matters

It shows a complete ORDB design workflow:

```
ER model -> choose object types -> choose inheritance -> choose REF links -> choose nested collections
-> create tables and constraints
```

13. Easy decision rules for mapping ER to ORDB

Use these exam-friendly rules.

Rule 1: Entity type

- entity -> object type + object table

Rule 2: ISA hierarchy

- supertype -> NOT FINAL
- subtype -> UNDER

Rule 3: 1:1 or N:1 relationship with key constraint

- use a REF attribute in the appropriate object type

Rule 4: M:N relationship or relationship with its own attributes

- use a separate object type/table

Rule 5: Weak entity with no independent references

- consider nested collection inside the owner

Rule 6: Participation constraint

- enforce with NOT NULL or reference constraints

14. Exam focus

You should be able to:

1. explain what a trigger is
2. read and explain a row-level trigger using :old and :new
3. map ER entities and relationships to ORDB types/tables
4. explain when to use inheritance, REF attributes, separate relationship types, and nested collections
5. justify why a weak entity might be modeled as a nested table

15. Common mistakes

- Forgetting that triggers can also work on object tables
- Reading :old and :new as tables instead of row versions
- Using inheritance without declaring the supertype NOT FINAL
- Using a REF for an M:N relationship that really needs a separate relationship type
- Using nested collections for weak entities that must be independently referenced elsewhere

16. One final memory sentence

Lecture 4A teaches how ORDB schemas are designed from ER models and how triggers automate responses to data changes.

Lecture 4B - XML Databases

Based on: XML Databases lecture slides.

1. Big idea

XML is used to represent **semi-structured data** using meaningful tags.

Databases can store and query XML either in native XML form or by integrating XML support into existing DBMS technology.

This lecture is really about three things:

1. what XML is
2. how XML is stored in databases
3. how XML is queried using XPath and XQuery

2. What XML is

XML stands for **eXtensible Markup Language**.

The lecture contrasts XML with HTML:

- HTML tags mainly describe formatting
- XML tags describe meaning or semantics

Example idea:

- HTML might say something should be italic
- XML might say something is a name, address, or product

Why this matters

XML is designed for **data representation and exchange**, not just display.

3. Well-formed XML

An XML document must at least be **well-formed**.

The lecture gives the main rules:

- start with XML declaration, usually `<?xml version="1.0"?>`
- have a single root element
- tags must be properly matched
- tags must be properly nested
- tags are case-sensitive

Example

```
<?xml version="1.0"?>
<BARS>
  <BAR>
    <NAME>Joe's Bar</NAME>
    <BEER>
      <NAME>Bud</NAME>
      <PRICE>2.50</PRICE>
    </BEER>
    <BEER>
      <NAME>Miller</NAME>
      <PRICE>3.00</PRICE>
    </BEER>
  </BAR>
</BARS>
```

Easy distinction

- **well-formed** -> syntactically correct XML
- **valid** -> conforms to a schema definition

4. XML schema languages

The lecture says schemas describe:

- structure of the XML document
- allowed tags
- nesting order
- multiplicity of subelements
- data content constraints

Schema languages mentioned:

- **DTD** (Document Type Definition)
- **XML Schema**
- **RELAX NG**

Purpose of a schema

A schema is used to validate whether an XML document has the expected structure.

5. Why XML storage matters

The lecture points out that XML became a de-facto standard for data exchange, so storing and querying large XML collections became important.

This created two main approaches.

6. Main approaches to XML storage

6.1 Native XML databases

Examples from the lecture:

- Timber
- Tamino
- Xindice

Characteristics:

- store XML in native XML form
- contain storage/retrieval techniques designed specifically for XML
- good when data is highly irregular or semi-structured

6.2 XML-enabled databases

Examples from the lecture:

- Oracle
- Microsoft SQL Server
- DB2

Characteristics:

- use existing DBMS technology to store and query XML
- benefit from mature database features developed over decades
- good when structure is fairly stable

6.3 When each approach is advantageous

The lecture gives a very important decision rule:

Situation	Better approach
schema is static / structured	XML-enabled DB

Situation	Better approach
schema is dynamic / semi-structured	Native XML DB

This is an excellent exam point.

7. XML-enabled storage options

The lecture mentions two main styles.

7.1 Map XML to existing DB structures

This means breaking XML into relational or object-relational structures.

7.2 Store XML as large objects

Examples:

- CLOB
- BLOB

This stores the XML more like document text/binary rather than a deeply queryable tree structure.

8. Native XML data types in DBMSs

Modern DBMSs often support a built-in XML datatype.

Examples in the lecture:

- Oracle XMLType
- SQL Server xml

The lecture notes that the internal representation preserves XML properties such as:

- containment hierarchy
- document order
- element values
- attribute values

This blurs the line between “native XML database” and “DBMS with XML datatype support”.

9. Typed vs untyped XML

This is one of the most important storage ideas.

9.1 Untyped XML

Use untyped XML when:

- you do not have a schema
- you have a schema but do not want the server to validate against it

Example in SQL Server:

```
CREATE TABLE AdminDocs (
  id INT PRIMARY KEY,
```

```
xDoc XML NOT NULL  
);
```

9.2 Typed XML

Use typed XML when:

- you have an XML schema collection
- you want validation against the schema
- you want query/storage optimizations based on type information

Example:

```
CREATE TABLE AdminDocs (  
  id INT PRIMARY KEY,  
  xDoc XML(CONTENT myCollection)  
);
```

Easy difference

Type	Meaning
untyped XML	database stores XML without schema-based typing/validation
typed XML	XML is associated with schema information

10. Inserting XML data

The lecture gives a sample insert:

```
INSERT INTO AdminDocs VALUES (  
  1,  
  '<book>  
    <title>Writing Secure Code</title>  
    <author>  
      <first-name>Michael</first-name>  
      <last-name>Howard</last-name>  
    </author>  
    <author>  
      <first-name>David</first-name>  
      <last-name>LeBlanc</last-name>  
    </author>  
    <price>39.99</price>  
  </book>'  
);
```

This is useful because it shows XML stored as a single value inside a database column.

11. How XML is queried

The lecture says there are two main ways:

1. **Path expressions / XPath**
2. **XQuery**

Rule of thumb

- use **XPath** when you mainly want to navigate and extract pieces
- use **XQuery** when you need a full query language similar in power to SQL

12. XPath basics

XPath describes paths through the XML tree.

12.1 Simple path descriptors

A path is made of tags separated by /.

- / at the start means start from the root
- // means the path may start anywhere in the document

Example

- /BARS/BAR/PRICE -> price elements directly along that root-based path
- //PRICE -> all price elements anywhere in the document

12.2 Wildcard *

* means any tag at that position.

Example:

- /*/*/PRICE -> all PRICE elements at the third nesting level

12.3 Attributes in XPath

Attributes are written with @.

Example:

- /BARS/*/ @name

This returns the name attributes of the immediate children of BARS.

13. Axes in XPath

The lecture explains that each XPath step can use an **axis**.

An axis says which direction to move in the XML tree.

Common axes

- child:: - children of the current node (default axis)
- attribute:: - attributes of the current node (@ is shorthand)
- parent:: - parent of the current node
- descendant-or-self:: - current node plus descendants
- ancestor:: - ancestors of the current node

Example

/BARS/BEER is shorthand for:

```
/BARS/child::BEER
```

and

```
@name
```

is shorthand for:

```
attribute::name
```

14. Predicates in XPath

A predicate is a condition written inside square brackets.

It filters nodes selected by the path.

Example: value condition

```
/BARS/BAR[PRICE < 2.75]/PRICE
```

Meaning:

- select BAR elements that have a PRICE less than 2.75
- then return their PRICE elements

Example: attribute condition

```
/BARS/BAR/PRICE[@theBeer = "Miller"]
```

Meaning:

- return price elements whose theBeer attribute is Miller

Position predicates

A number in a predicate can mean child position.

Examples:

- /BARS/BAR[1] -> first BAR
- /BARS/BAR[2]/PRICE[1] -> first price under second bar

The lecture also notes that multiple predicates can be combined and are evaluated left to right.

15. XQuery

XPath is good for navigation. XQuery is the fuller language for querying XML.

The lecture says:

XQuery extends XPath to a query language with power similar to SQL.

15.1 Variables

XQuery variables start with \$.

Example:

```
$doc  
$bar  
$price
```

15.2 FLWOR expression

The most important XQuery pattern is **FLWOR**:

- **F** - for
- **L** - let
- **W** - where
- **O** - order by
- **R** - return

It is the XML equivalent of the way SQL uses FROM, WHERE, ORDER BY, and SELECT.

What each clause does

for

Iterates through sequences of nodes or values.

let

Binds a variable to a value or expression without iterating.

where

Filters the tuples/bindings.

order by

Sorts the result.

return

Constructs the final output.

15.3 Easy FLWOR example

```
for $b in doc("bars.xml")/BARS/BAR
where $b/PRICE < 2.75
order by $b/@name
return $b/NAME
```

Plain meaning:

- go through each bar
- keep only bars with a cheap price
- sort by bar name
- return the bar name

15.4 Example that constructs XML output

```
for $a in doc("books.xml")/book/author
return <authorName>{ $a/first-name } { $a/last-name }</authorName>
```

This shows that XQuery can return new XML, not just plain scalar values.

15.5 Concepts named in the lecture slides

The later slides mention several FLWOR-related ideas. You should know the meaning of each:

- **range expression** - expression used in for to define the sequence to iterate over
- **multiple variables in a for clause** - allows nested iteration
- **positional variables** - track the position of an item in the sequence
- **multiple for and let clauses** - support more complex binding logic
- **variable binding and referencing** - variables hold nodes/values and are reused later
- **comparisons** - compare nodes, values, or sequences
- **conditional expressions** - if ... then ... else ...
- **effective boolean value** - how XQuery interprets values in conditions
- **functions** - built-in functions and user-defined logic

Even if the slides only listed these as headings, they are part of the XQuery toolbox and are common exam topics.

16. SQL Server methods on XML datatype

The lecture ends with practical methods available on SQL Server's XML type.

16.1 query()

Returns XML fragments.

```
SELECT xDoc.query('/doc[@id = 123]//section')
FROM AdminDocs;
```

Use when you want part of the XML back as XML.

16.2 value()

Returns a scalar value converted to an SQL type.

```
SELECT xDoc.value(
    'data(/doc//section[@num = 3]/title)[1]',
    'nvarchar(max)'
)
FROM AdminDocs;
```

Use when you want a single extracted value.

16.3 exist()

Checks whether a path expression matches anything.

```
SELECT xDoc.query('/doc[@id = 123]//section')
FROM AdminDocs
WHERE xDoc.exist('/doc[@id = 123]') = 1;
```

Use when you want a yes/no style condition.

16.4 modify()

Changes XML content using XML DML.

```
UPDATE AdminDocs
SET xDoc.modify('
  insert
    <section num="2">
      <title>Background</title>
    </section>
  after (/doc//section[@num=1])[1]
');
```

This lecture mentions that XML DML supports operations such as:

- insert
- delete
- update

17. XPath vs XQuery

Feature	XPath	XQuery
Main purpose	navigation/path selection	full query language for XML
Output	selected nodes/values	values, nodes, or constructed XML
Power level	simpler	richer and closer to SQL
Best use	locate parts of a document	complex querying and transformation

18. Exam focus

Be able to explain:

1. what XML is and why it is useful
2. well-formed vs valid XML
3. native XML DB vs XML-enabled DB
4. typed vs untyped XML
5. how XPath paths, axes, attributes, and predicates work
6. what FLWOR means in XQuery
7. what query(), value(), exist(), and modify() do

19. Common mistakes

- Confusing XML with HTML
- Thinking well-formed and valid mean the same thing
- Forgetting that / means root-based path while // means anywhere in the document
- Forgetting @ for attributes
- Mixing XPath navigation with XQuery construction logic
- Using query() when you really need a scalar from value()

20. One final memory sentence

Lecture 4B teaches how XML stores semi-structured data and how databases query that data using XPath and XQuery.

Lecture 5 - Components of a DBMS, Disks, and Files

Based on: Components of a DBMS, Disks and Files lecture slides.

1. Big idea

This lecture moves from data modeling to the physical side of database systems.

The main message is:

DBMS performance is dominated by how data is stored on disk and moved into main memory.

The lecture explains:

- major DBMS components
- why disks matter so much
- how the DBMS manages disk space and buffers
- why storage organization strongly affects performance and reliability

2. Main components of a DBMS

The lecture gives a high-level DBMS structure.

2.1 Query processing path

A DBMS accepts SQL queries, then:

1. parses them
2. sends them to the optimizer
3. creates an execution plan
4. executes the plan using relational operators and storage mechanisms

2.2 Storage-related components

The lecture names these components:

- **Files and access methods** - interface to manipulate stored files
- **Buffer manager** - brings pages from disk into memory
- **Disk space manager** - handles space allocation/deallocation on disk

2.3 Transaction-related components

The lecture also mentions transaction processing modules:

- **Transaction manager** - ensures lock requests/releases follow protocol
- **Lock manager** - tracks locks on database objects
- **Recovery manager** - maintains logs and restores consistency after crashes

Easy structure to remember

Area	Components
Query processing	parser, optimizer, execution plan
Storage	files/access methods, disk space manager, buffer manager
Transactions/recovery	transaction manager, lock manager, recovery manager

3. Why not store everything in main memory?

The lecture asks an excellent question: why not keep the whole database in RAM?

Reasons

1. **Cost** - main memory is more expensive than disk
2. **Volatility** - RAM loses contents when power is lost
3. **Size** - databases are usually larger than available RAM

Storage hierarchy

The lecture shows the hierarchy:

- **main memory** for currently used data
- **disk** for the main database
- **tape** for archival storage

General principle

As we move downward in the hierarchy:

- cost per unit decreases
- speed decreases
- capacity usually increases

4. Disks as the main secondary storage device

The lecture says disks are the storage device of choice because:

- they support random access
- unlike tapes, they do not require purely sequential access

But disks are much slower than memory, so the DBMS must minimize disk I/O.

5. Disk structure

A magnetic disk contains:

- **platters** - circular surfaces that spin
- **spindle** - central axis
- **tracks** - concentric circles on a platter
- **sectors** - subdivisions of tracks

- **disk head** - reads/writes data
- **arm assembly** - moves the head in/out
- **cylinder** - set of aligned tracks across platters

Why this matters

The physical location of a block affects access time.

6. Disk page access time

The lecture breaks access time into three parts:

Access time = seek time + rotational delay + transfer time

6.1 Seek time

Time to move the disk arm so the head reaches the correct track.

6.2 Rotational delay

Time waiting for the desired sector/block to rotate under the head.

6.3 Transfer time

Time to actually read/write the data.

Important observation from the lecture

Seek time and rotational delay dominate the cost.

That means many performance ideas aim to reduce:

- unnecessary movement
- unnecessary waiting for rotation

7. Why page placement matters

The lecture introduces the idea of the “next” block:

- same track first
- then same cylinder
- then adjacent cylinder

If blocks of a file are laid out sequentially on disk, sequential scans become much faster.

Prefetching

For sequential scans, it is useful to prefetch several pages at once into memory.

This reduces waiting and improves throughput.

8. RAID

Disks are a performance bottleneck and are not perfectly reliable, so DBMS environments often use **RAID**.

RAID = Redundant Array of Independent Disks.

RAID goals

- increase performance
- increase reliability

Two main ideas in RAID

1. **Data striping**
 - spread data across multiple disks
2. **Redundancy**
 - store extra information so data can be recovered if a disk fails

9. Why striping improves performance

If data is spread over D disks, then up to D blocks can be transferred in parallel.

So the aggregate transfer rate can increase significantly.

10. Why redundancy is needed

More disks mean more chances that some disk will fail.

So a disk array without redundancy can actually reduce overall reliability.

That is why parity or mirroring is added.

11. RAID levels

The lecture summarizes several RAID levels. You should know their basic ideas.

RAID level	Core idea	Main strength	Main weakness
RAID 0	striping only, no redundancy	highest write performance	no fault tolerance
RAID 1	mirroring	very high reliability, good reads	expensive, 50% space utilization
RAID 0+1	striping + mirroring	speed + redundancy	costly, 50% utilization
RAID 2	bit-level striping + Hamming code	historical error-correction design	rarely emphasized in practice
RAID 3	bit-interleaved parity	good for large sequential I/O	all disks involved in each request
RAID 4	block-interleaved parity with dedicated check disk	parallel small reads possible	parity disk can become a bottleneck
RAID 5	distributed block parity	balanced practical choice	writes still pay parity overhead
RAID 6	like RAID 5 with extra redundancy	survives two disk failures	more overhead

Most exam-friendly descriptions

- **RAID 0** -> fast but unsafe
- **RAID 1** -> mirror copies, expensive but reliable
- **RAID 5** -> distributed parity, common compromise
- **RAID 6** -> can survive two failures

12. Disk space management

The disk space manager is the lowest storage layer of the DBMS.

It provides operations such as:

- allocate a page
- deallocate a page
- read a page
- write a page

Important lecture point

Higher DBMS layers do not need to know the low-level details of how free space is managed.

Ways to track free space

The lecture gives two classic methods:

- free list of blocks
- bitmap of free/used blocks

Sequential allocation

If a sequence of pages is requested, the system should try to allocate them sequentially on disk whenever possible.

This helps reduce seek and rotational costs.

13. Buffer management

A DBMS cannot operate on data that remains only on disk.

Data must first be brought into RAM, into the **buffer pool**.

13.1 Buffer pool idea

- disk pages are copied into memory frames
- the DBMS tracks which page is in which frame
- pages are reused as needed

13.2 Important metadata per frame

The lecture gives two essential values:

- **pin count** - how many users/operators are currently using the page
- **dirty bit** - whether the page has been modified since it was read from disk

14. What happens when a page is requested?

The lecture gives the standard workflow.

Case 1: page already in buffer pool

- return it immediately

Case 2: page not in buffer pool

1. choose a frame for replacement
2. if the chosen frame is dirty, write it back to disk
3. read the requested page into the frame
4. pin the page
5. return its address

This is one of the most important operational patterns in DBMS internals.

15. Unpinning and replacement eligibility

When an operation finishes with a page, it must **unpin** it.

A page can be replaced only when:

```
pin count = 0
```

This is a very important rule.

Why?

Because replacing a page that is still being used would break the operation using it.

16. Buffer replacement policies

When the buffer is full, the DBMS chooses a page/frame to replace.

Policies mentioned in the lecture:

- LRU (Least Recently Used)
- Clock
- MRU (Most Recently Used)
- Random

Key point

The best policy depends on the workload.

17. Sequential flooding

The lecture highlights a classic problem: **sequential flooding**.

This happens when:

- a file is repeatedly scanned sequentially
- the file has more pages than the available buffer frames
- LRU keeps evicting pages in an unhelpful way

In such a case, every page request may cause an I/O.

The lecture notes that **MRU may be better** for this specific pattern.

This is a famous example showing that no single replacement policy is always best.

18. DBMS vs OS file system

A natural question is: why not let the operating system do all file and buffer management?

The lecture gives several reasons.

DBMS needs extra control

- pin pages in memory
- force pages to disk when required
- adjust replacement policy based on DB workload
- prefetch pages intelligently
- integrate storage behavior with concurrency control and recovery rules

So the DBMS cannot completely rely on a generic OS file system policy.

19. Exam focus

You should be able to explain:

1. the main components of a DBMS
2. why disk I/O dominates cost
3. the parts of disk access time
4. the purpose of RAID and differences among key RAID levels
5. what disk space management does
6. how the buffer manager works
7. what pin count and dirty bit mean
8. why DBMS needs control beyond the OS file system

20. Common mistakes

- Saying transfer time is the main cost on traditional disks; the lecture emphasizes seek and rotational delay
- Forgetting that pages must be in RAM before the DBMS can process them
- Forgetting that dirty pages must be written back before replacement
- Mixing up pin count and dirty bit
- Assuming LRU is always best
- Confusing RAID performance goals with reliability goals

21. One final memory sentence

Lecture 5 teaches the physical foundation of DBMS performance: pages, disks, buffers, and reliability mechanisms.

Lecture 6 - File Organization and Indexes

Based on: File Organization and Indexes lecture slides.

1. Big idea

This lecture explains how records are physically organized on disk and how indexes make searching efficient.

The central lesson is:

How data is organized physically determines how expensive search, insert, delete, and range queries become.

2. Files of records

A DBMS reads and writes **pages**, but higher layers usually think in terms of **records**.

So a file is:

- a collection of pages
- each page contains records

A file structure must support:

- insert record
- delete record
- modify record
- retrieve a particular record by record id
- scan all records, optionally with conditions

3. Main file organization types

The lecture discusses three major file organizations:

- heap (unordered) files
- sequential/sorted files
- hashed files

4. Heap files

A **heap file** stores records in no particular order.

Strengths

- simplest design
- very good when typical access is a full file scan
- inserts are easy, often at the end or in free space

Weaknesses

- equality search requires scanning unless an index exists
- range search requires scanning
- delete may require search first

Easy memory sentence

Heap = easiest to maintain, weakest for direct searching.

5. Sorted (sequential) files

A sorted file stores records in order of a chosen search key.

Strengths

- efficient equality search
- efficient range search
- good when output must be in order

Weaknesses

- insert is expensive because correct position must be found and records/pages may need to move
- delete may also require movement to preserve order

Easy memory sentence

Sorted file = great for search and range, costly for maintenance.

6. Hashed files

A hashed file uses a hash function to place records into buckets.

Basic idea

- file is divided into buckets
- $h(r)$ decides which bucket record r belongs to
- each bucket may have a primary page and overflow pages

Strengths

- very good for equality search on the hash key

Weaknesses

- poor for range search
- performance can degrade when overflow chains become long

Easy memory sentence

Hashed file = best for exact-match search, poor for ranges.

7. Qualitative comparison of file organizations

Operation	Heap	Sorted	Hashed
Full scan	good	good	good
Equality search	poor unless indexed	good	very good on hash key
Range search	poor	very good	poor
Insert	easy	expensive	usually good until overflow grows

Operation	Heap	Sorted	Hashed
Delete	moderate, often requires search	moderate to expensive	good if bucket known

8. Heap implementation details

The lecture describes two common ways to organize heap files.

8.1 Heap file as a linked list of pages

There may be:

- header page
- pages with free space
- full pages

Problem:

- finding a page with enough free space may require scanning many pages

8.2 Page directory

A better approach is a **page directory**.

Directory entries store information about pages, such as available free bytes.

Advantage

A page directory is much smaller than the full set of data pages, so finding suitable space becomes more efficient.

9. What is an index?

An index speeds up selection on chosen search-key fields.

Important lecture point:

- **search key** is not the same as **primary key**
- any subset of fields can be used as an index search key

An index is a separate data structure that helps locate records faster.

10. Costs and benefits of indexes

Benefits

- faster access to qualifying records

Costs

- takes extra space
- inserts, updates, and deletes may become slower because index entries must also be maintained

This trade-off is a standard exam point.

11. Alternatives for index data entries

The lecture gives three alternatives for a data entry k^* in an index.

Alternative 1

Store the actual data record with key value k .

Alternative 2

Store:

```
<k, rid of record>
```

Alternative 3

Store:

```
<k, list of rids>
```

Why this matters

Different access methods and storage designs choose different entry styles, affecting space use and lookup behavior.

12. Clustered vs unclustered indexes

This is one of the most important theory distinctions.

12.1 Clustered index

The data records are stored in the same order as the index key order, or very close to it.

Effect

If nearby index entries refer to nearby data pages, many records can be fetched with fewer I/Os.

12.2 Unclustered index

The data records are not stored in the same order as the index.

Effect

Following index entries may jump to many scattered pages, making retrieval more expensive.

Important lecture rule

A table can have at most **one clustered index**, because the data can only be physically ordered one main way.

13. Dense vs sparse indexes

13.1 Dense index

At least one index entry exists for every search-key value in the data.

Alternative 1 always gives a dense index.

13.2 Sparse index

Index entries exist only for some search-key values.

Important lecture rule

Every sparse index is **clustered**.

Why?

Because sparse indexing depends on data order to allow navigation from a few representative entries.

Trade-off

- sparse index is smaller
- dense index may support more direct access patterns

14. Primary, secondary, and unique indexes

- **Primary index** - search key contains the primary key
- **Secondary index** - any other index
- **Unique index** - search key contains a candidate key so each key value identifies at most one record

15. Composite search keys

An index can be built on multiple fields together.

Example search key:

```
<sal, age>
```

The order is lexicographic.

That means:

- first compare sal
- if equal, compare age

Why order matters

An index on <sal, age> is not the same as an index on <age, sal>.

Equality query example

```
age = 20 AND sal = 75
```

Range query example

```
age = 20 AND sal > 10
```

For composite keys, the usefulness of an index depends heavily on the order of fields and the type of condition.

16. Index support in SQL

The lecture notes that indexes are not part of original SQL-92, but practical DBMSs provide statements such as:

- CREATE INDEX
- DROP INDEX

So indexes are a real implementation feature, even if relational theory often discusses them separately.

17. Range search idea

If a file is sorted, binary search can find the first matching record for a range and then scan from there.

But binary search directly on a big data file can still be expensive.

That motivates a smaller **index file** that can be searched quickly.

18. B+ trees

The lecture identifies the **B+ tree** as the most widely used index structure.

18.1 Why B+ trees are popular

- balanced tree
- efficient equality search
- efficient range search
- insert/delete handled dynamically
- good fanout means small tree height

18.2 Structure

A B+ tree has:

- internal index nodes
- leaf nodes containing data entries
- leaves linked in sequence order (the sequence set)

Important property

All search paths from root to leaf have the same length.

So the tree remains height-balanced.

18.3 Occupancy rule

The lecture states that, except for the root, each node has between d and $2d$ entries.

This guarantees at least 50% occupancy.

19. B+ tree search

Search starts at the root.

At each internal node, key comparisons decide which pointer/subtree to follow.

Eventually the search reaches the correct leaf.

Why range queries are efficient

Once the first matching leaf entry is found, leaf pages can be scanned in order.

That is something hashing cannot do well.

20. Insertion into a B+ tree

The lecture gives the standard pattern.

1. find the correct leaf
2. insert the new data entry
3. if the leaf has room, stop
4. if the leaf overflows, split the leaf
5. copy up the middle key to the parent
6. if the parent overflows, split recursively
7. if the root splits, tree height increases by one

Very important distinction

- **leaf split** -> middle key is **copied up**
- **internal node split** -> middle key is **pushed up**

This is a classic exam point.

21. Deletion from a B+ tree

Standard pattern:

1. find the leaf containing the entry
2. delete the entry
3. if occupancy still okay, stop
4. if underfull, try redistribution from sibling
5. if redistribution fails, merge with sibling
6. delete/update parent entry accordingly
7. if root becomes too small, tree height may decrease

Terms to remember

- **redistribution** - borrow entries from sibling
- **merge** - combine two nodes

22. Duplicates in B+ trees

The lecture notes possible approaches:

- overflow leaf pages
- duplicate values directly in leaf pages
- make keys unique using rowids

Many systems prefer adding rowid information so that duplicate key values become distinguishable.

23. Hash-based indexing

Hash-based indexes are best for **equality selections**.

They do **not** support range searches well.

The lecture discusses both static and dynamic hashing.

24. Static hashing

In static hashing:

- number of primary buckets is fixed
- buckets are allocated sequentially
- overflow pages are used if needed
- hash function often looks like $h(\text{key}) \bmod N$

Problem with static hashing

As inserts continue:

- overflow chains can become long
- performance degrades
- deletions may waste space

This is why dynamic hashing was developed.

25. Extendible hashing

Extendible hashing solves the growth problem more gracefully.

Main idea

- maintain a **directory** of pointers to buckets
- use hash-bit prefixes/suffixes to choose a bucket
- when a bucket overflows, split only that bucket
- if needed, double the directory

Key terms

- **global depth** - number of hash bits used by the directory
- **local depth** - number of bits relevant to a specific bucket

Important rule from the lecture

Directory doubling happens when splitting a bucket requires more bits than the current global depth provides.

In other words, when the overflowing bucket's local depth becomes greater than the global depth, the directory must grow.

26. Comments on extendible hashing

The lecture gives several practical points:

- if the directory fits in memory, equality search is very fast
- directory growth is cheaper than reorganizing the whole file
- skewed hash values can still create problems
- delete may allow bucket merge and possible directory halving

27. B+ tree vs hashing

This is one of the most useful exam comparisons.

Feature	B+ tree	Hashing
Equality search	very good	excellent
Range search	very good	poor
Ordered output	yes	no
Dynamic balancing	yes	depends on scheme
Typical best use	mixed workloads, ranges, ordered retrieval	exact-match lookups

Easy memory sentence

- **B+ tree** -> flexible and ordered
- **hashing** -> exact-match specialist

28. Exam focus

You should be able to explain:

1. the difference between heap, sorted, and hashed file organization
2. page directory vs simple linked-page heap organization
3. what an index is and why it helps
4. clustered vs unclustered, dense vs sparse, primary vs secondary
5. how B+ tree search/insert/delete work conceptually
6. why B+ trees support range queries well
7. how static hashing differs from extendible hashing
8. when hashing is preferred over B+ trees and vice versa

29. Common mistakes

- Confusing search key with primary key
- Saying multiple clustered indexes can exist on the same table
- Forgetting that every sparse index is clustered

- Forgetting the difference between copy-up and push-up in B+ tree splits
- Saying hash indexes are good for range queries - they are not
- Confusing static hashing overflow pages with extendible hashing directory-based growth

30. One final memory sentence

Lecture 6 teaches that physical file organization and indexing decide whether database operations are cheap or expensive.

Final cross-lecture revision sheet

1. The module in one page

ORDB side

- user-defined types model complex objects
- collections model one-to-many relationships inside objects
- methods add behavior
- inheritance models hierarchies
- triggers automate actions
- ER-to-ORDB mapping tells you how to design the schema

XML side

- XML models semi-structured tagged data
- XPath navigates XML
- XQuery queries and constructs XML
- databases can store XML natively or through XML-enabled support

Storage and indexing side

- disk I/O is the major cost driver
- buffer management reduces disk reads/writes
- heap/sorted/hashed files affect access cost
- indexes speed search at extra maintenance cost
- B+ trees are general-purpose; hashing is best for equality search

2. High-value comparison table

Topic	Know this contrast
row object vs column object	whole row is an object vs column holds an object
VARRAY vs nested table	bounded ordered list vs table-like child collection
map vs order method	compare via scalar mapping vs compare object-to-object
final vs not final	no subtypes vs subtypes allowed
overriding vs overloading	new implementation vs same name different signature
REF vs foreign key-style thinking	object pointer semantics vs classic relational link

Topic	Know this contrast
native XML DB vs XML-enabled DB	XML-first storage vs DBMS-first integration
typed vs untyped XML	schema-aware vs schema-agnostic
heap vs sorted vs hashed files	easy insert vs good ranges vs good equality
clustered vs unclustered index	data near key order vs data scattered
B+ tree vs hashing	supports ranges vs best for equality

3. Final exam strategy

When answering a theory question:

1. **define the concept clearly**
2. **state why it is used**
3. **give the structure or syntax pattern**
4. **give one small example**
5. **state the advantage / limitation**
6. **compare with the closest alternative if relevant**

That structure works for almost every topic in these lectures.

4. Final memory line for the whole set

The first half of the notes is about richer ways to model data; the second half is about how the DBMS stores and finds that data efficiently.